Third Handson Report

Simone Stanganini mat. 616834

December 8, 2025

- **Problem 1: Holiday Planning**

  For this implementation, initially i approached the problem using a DAG, but during implementation i found it difficult to manage in those terms so i switched to a dynamic programming approach based on `prefix_sum` (initially i didn't use it, but to reduce complexity i realized it was enough to compute the `prefix_sum` at the start) and also, initially i used a table for maximums, but then i realized that only two arrays were needed.

  First, i calculate the `prefix_sum` of the number of attractions for each day in every city, then i use two arrays, `max` and `new_max`. The item `max[i]` holds the best attraction value for the `i` day, while `new_max` contains the newly calculated maximums.

  I iterate through every day in the `prefix_sum` table and try to update the `new_max` array by looking for a combination between the current days and the values in the `max` array.

  This is done using a loop over all elements (the two loops for n and d) and an inner loop from $j-1$ to 0 to check all combinations. Therefore, in the worst case, the complexity is $\Theta(n \cdot d^2)$ since for every j we iterate at most d times. The space cost of the algorithm is linear, $\Theta(n \cdot d)$.

- **Problem 2: Desing Course**

  Tackling this problem, its clear that we have to transform it into other terms, meaning ordering the items in a way to find the right combination. To do this i decided to sort the items by `difficulty` and find the LIS (Longest Increasing Subsequence) among the `beauty` values.

  To write the LIS algorithm, after reading the professor's Dynamic Programming notes on GitHub, instead of doing binary search on an array i decided to implement a custom Binary Search Tree:

  – I check the maximum value in the BST. if the current course's beauty is greater than the maximum, i simply add it to the tree (extending the length).

  – If it is not greater, i search for its successor (the smallest element in the BST that is greater than or equal to the current key) and replace it, this maintains the property required for the LIS algorithm.

  The initial sorting takes $\Theta(n \log n)$. then i perform n insertions/searches in the BST, and each operation costs on average $\Theta(\log n)$, so the total time complexity is $\Theta(n \log n)$. The space complexity is $\Theta(n)$ to store the nodes of the tree.